PAYware Wynid

Verifone

WCL CB5

BASE DLL

INTERFAÇAGE DE LA MONETIQUE

Service : **Service Intégration**
Projet : **BASE DLL**
Version : **2.50-FR-r1**
Date : **31/03/2017**

BUREAUX

Verifone Siège - *Headquarters*

Verifone Systems France
43, avenue de l'Europe
78140 Vélizy Villacoublay
France
Tél : +33 1 39 45 12 10

Verifone Systems France
Wynid et support clients
Immeuble le Mustang
144, Rue d'Odin
34000 Montpellier
France
Tel: + 33 (0) 4 67 66 53 66

Verifone Systems France | PTS
11 A, rue Jacques Cartier
78280 Guyancourt
France
Tel: + 33 (0)1 61 37 05 70
Fax: + 33 (0)1 61 38 16 56

Verifone Systems France | PTS
Services Techniques et Supports Clients
Centre Espace Sud - Avenue d'Elne
66200 Latour-bas-Elne
France
Tel: + 33 (0)4 68 85 79 90
Fax: + 33 (0)4 68 85 79 99

Verifone Systems France | PTS
Maintenance et logistique
ZAC BOIS CHALAND 2
9, Rue des Pyrénées
91090 Lisses
France
Tél: 0820 01 9000
Fax: 0820 000 501

LISTE DE DIFFUSION

Nom	Société et/ou Service	Fonction	Mail
	INTÉGRATION	Chef de Projet	
	INTÉGRATION	Chef de Projet	
	BUREAU D'ÉTUDE	Chef de Projet	

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

TABLEAU DES RÉVISIONS

Index	Date	Commentaire	Auteur
2.50-FR-r1	31/03/2017	Refonte du document avec le **Nouveau Template** **VERIFONE**	INTEGRATION
2.42	31/07/2015	Mise à jour schémas	
2.41	08/01/2015	Corrections	INTEGRATION
2.40	03/11/2014	New Logo	INTEGRATION
2.33	11/06/2013	Corrections sur pointeurs variables	INTEGRATION
2.32	14/11/2012	Mise au format **PAYWare Wynid** / **VERIFONE**	INTEGRATION
2.31	17/01/2012	Correction texte `TimeOutMessage` du `SwimApiOpen`	INTEGRATION
2.30	03/01/2008	Ajout informations mode “echo”	INTEGRATION
2.20	22/05/2006	Correction de l’exemple d’identification des traces	INTEGRATION
2.10	18/05/2006	Identification des traces générées par la dll	INTEGRATION
2.00	13/03/2006	Ajout d’un schéma type de l’interfaçage de la monétique Modification de la description des paramètres d’appel des fonctions de la dll Ajout du code retour « `SWIMAPI_ERROR_CHRONO` » en réponse à la fonction « `SWimApiSendMessage` »	INTEGRATION
1.00	26/01/2005	Création du document	INTEGRATION

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

TABLE DES MATIERES

INTRODUCTION 1
1. LÉGENDE 1
1.1. format des données transportées 1
1.2. conditions de présence des données 2
2. DOCUMENTS DE RÉFÉRENCES 3
DÉFINITIONS 5
1. PRÉSENTATION 5
PRINCIPES 7
1. DÉMARRAGE DU CLIENT MONÉTIQUE 7
2. ARRÊT DU CLIENT MONÉTIQUE 7
3. ÉCHANGE DE MESSAGES AVEC LE CLIENT MONÉTIQUE 7
4. EXEMPLE D'ÉCHANGES 8
5. FORMATAGE DES REQUÊTES ET DES RÉPONSES 8
5.1. Format 8
5.2. exemple de message 9
5.3. Construction d'une requête 9
5.4. Extraction des informations d'une réponse 10
SCHÉMA TYPE DE L'INTERFACE MONÉTIQUE 11
1. SCHÉMA 11
CINÉMATIQUE DE L'INTERFACE MONÉTIQUE 13
1. CINÉMATIQUE 13
IDENTIFICATION DES TRACES 15
1. TRACES DLL 16
2. TRACES MCL 17
FONCTIONS D'ÉCHANGES AVEC LE CLIENT MONÉTIQUE 19
1. FONCTION « SWIMAPIOPEN » 19
1.1. fonctionnalités 19
1.2. paramètres d'appel de la fonction 19
1.3. codes résultat 22
2. FONCTION « SWIMAPICLOSE » 23
2.1. fonctionnalités 23
2.2. paramètres d'appel de la fonction 23
2.3. codes résultat 23
3. FONCTION « SWIMAPISENDMESSAGE » 24
3.1. fonctionnalités 24
3.2. paramètres d'appel de la fonction 24
3.3. codes résultat 25
4. FONCTION « SWIMAPIWAITSYNCHRO » 28
4.1. fonctionnalités 28

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

4.2. paramètres d'appel de la fonction 28
4.3. codes résultat 29
5. FONCTION « SWIMAPIGETMESSAGE » 30
5.1. fonctionnalités 30
5.2. paramètres d'appel de la fonction 30
5.3. codes résultat 31
FONCTIONS DE CRÉATION D'UN MESSAGE 33
1. FONCTION « ATAG_WRITEINIT » 33
1.1. fonctionnalités 33
1.2. paramètres d'appel de la fonction 33
1.3. codes résultat 33
2. FONCTION « ATAG_SETSTRINGVAL » 34
2.1. fonctionnalités 34
2.2. paramètres d'appel de la fonction 34
2.3. codes résultat 34
FONCTIONS D'ANALYSE D'UN MESSAGE 35
1. FONCTION « ATAG_READINIT » 35
1.1. fonctionnalités 35
1.2. paramètres d'appel de la fonction 35
1.3. codes résultat 35
2. FONCTION « ATAG_GETSTRINGVAL » 36
2.1. fonctionnalités 36
2.2. paramètres d'appel de la fonction 36
2.3. codes résultat 37

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

INTRODUCTION

Ce document décrit les fonctionnalités d'intégration de la solution **PAYware Wynid**.

1. LEGENDE

1.1. FORMAT DES DONNEES TRANSPORTEES

Les « **données »** sont transportées :

- au format ASCII,
- encapsulées dans un format propriétaire : ***{Etiquette}Valeur***

  - Les « ***Etiquettes***» :
    - sont fixes
    - **{SS}, {PS} et {F}** sont mises en place automatiquement par la **DLL (wcl_dll.cll)** lors de l'appel à la fonction **SWimApiSendMessage**

  - Les « ***Valeurs***» possèdent un format particulier par étiquette, de type :
    - **AnsX** : Donnée codée sur X octets ASCII (terminée par un nul)
    - **Ans..Y** : Donnée codée sur 1 à Y octets ASCII (terminée par un nul)
    - **AnsX..Y** : Donnée codée sur X à Y octets ASCII (terminée par un nul)

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

1.2. CONDITIONS DE PRESENCE DES DONNEES

Le tableau, ci-dessous, décrit les **conditions de présence des données** :

Présence	Condition
M	Donnée obligatoire dans le message
N	Donnée nécessaire mais non obligatoire (pas d'erreur retournée si absente, mais possible erreur sur ticket)
O	Donnée facultative dans le message
O (1)	Donnée facultative, obligatoirement présente dans le message en cas de résultat positif
R	Donnée obligatoire, répétable dans le message
I	Données **non nécessaires** – ignorée par le traitement de l'application mais conservée dans les traces

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

2. DOCUMENTS DE REFERENCES

Le tableau, ci-dessous, regroupe les documentations et spécifications sur lesquelles se réfère ce document :

- Documentations et spécifications du GIE et des émetteurs.
- Documentations internes VERIFONE.

Désignation	Rédacteur	Version	Date
Manuel Utilisateur WEB-NAVBD12	VERIFONE	2.0.0	10/12/2009
Manuel de Paiement sur Automate MPA 5.2.2	GIE CB	5.2.2	10/2000
Mise à jour du référentiel CB5.2 – Bulletin n°3 – Téléchargement de logiciel	GIE CB		03/2003
Mise à jour du référentiel CB5.2 – Bulletin n°6 – Exigences sécuritaires Paiement de Proximité et Paiement sur Automate	GIE CB		01/2005
Mise à jour du référentiel CB5.2 – Bulletin n°7 – Modalités d'implémentation de PCI PED dans les solutions CB5.2	GIE CB		06/2005
Mise à jour du référentiel CB5.2 – Bulletin n°8 – Communications avec le système d'acceptation paiement	GIE CB		12/2005
Mise à jour du référentiel CB5.2 – Bulletin n°10 – Télécollecte Paiement de Proximité et Paiement sur Automate	GIE CB		06/2007
Multi-application : Ergonomie des terminaux – Bulletin n°11 – Paiement de Proximité et Paiement sur Automate	GIE CB		02/2008
Bulletin n°13	GIE CB	1.0.0	03/2010
Précision Bulletin n°13	GIE CB		
CB2A Autorisation – Version 1.1.2	GIE CB	1.1.2	04/2003
CB2A Télécollecte Téléparamétrage Gestion de Réseau – Version 1.1.3	GIE CB	1.1.3	04/2003
CB2A Télécollecte Téléparamétrage Gestion de Réseau – Services additionnels – Évolution communications Accepteurs-Acquéreurs	GIE CB	1.1.6	11/2005

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

Page blanche laissée intentionnellement

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

DEFINITIONS

1. PRESENTATION

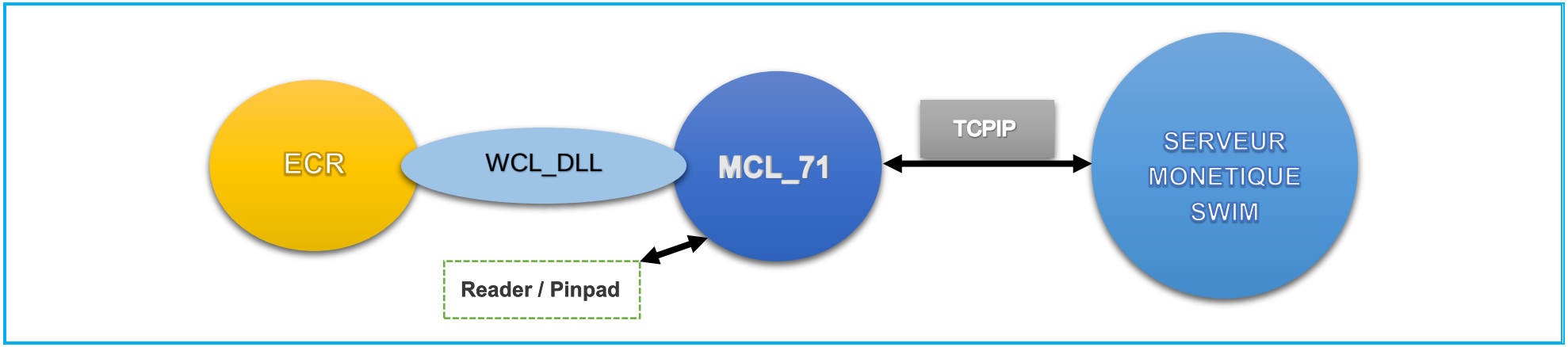
La solution monétique **Wynid** est caractérisée par trois éléments :

Serveur monétique	**SWIM**
Lecteur de cartes	**Lecteur PinPad VERIFONE**
Client monétique	**MCL_71**

*Le serveur monétique **SWIM*** effectue l'ensemble des contrôles sur les transactions, gère les appels vers les centres d'autorisations bancaires, stocke les opérations et prend en charge les remises bancaires.

Le lecteur de carte **VERIFONE** est le périphérique utilisé par le serveur monétique pour le dialogue avec les microcircuits des cartes à puces, à pistes **ISO** ou **NFC**.

*Le client monétique **MCL_71*** est la passerelle entre le serveur monétique **SWIM**, le lecteur de cartes **VERIFONE** et l'application de vente **POS** (**ECR**).
Il se présente sous forme d'un exécutable nommé **MCL_71.EXE** et d'une librairie d'interface avec l'application de vente **WCL_DLL.DLL**.



© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

L'application de vente dialogue avec le client monétique **MCL_71** par l'intermédiaire des fonctions ci-après de la librairie d'interface :

SWimApiOpen	Démarrage du client monétique
SWimApiClose	Arrêt du client monétique
SWimApiSendMessage	Émission d'un message à destination du client monétique
SWimApiWaitSynchro	Attente de disponibilité d'un message en provenance du client monétique
SWimApiGetMessage	Réception d'un message en provenance du client monétique

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

PRINCIPES

1. DEMARRAGE DU CLIENT MONETIQUE

L'application de vente est à l'initiative du démarrage du client monétique par l'appel de la fonction **« SWimApiOpen »** de la librairie d'interface.

Le client monétique peut être démarré soit au lancement de l'application de vente, soit à chaque début d'opération.

2. ARRET DU CLIENT MONETIQUE

L'application de vente est à l'initiative de l'arrêt du client monétique par l'appel de la fonction **« SWimApiClose »** de la librairie d'interface.

Le client monétique peut être arrêté soit :

- à l'arrêt de l'application de vente (cas où il est démarré en même temps que l'application de vente),
- à chaque fin d'opération (cas où il est démarré à chaque début d'opération).

3. ECHANGE DE MESSAGES AVEC LE CLIENT MONETIQUE

Le **processus d'échanges** avec le **client monétique** est du type **Requête / Réponse**.

L'application de vente est à l'initiative du premier message échangé par l'appel de la fonction **« SWimApiSendMessage » (Requête)**.

L'application de vente est ensuite à l'écoute du client monétique par l'appel de la fonction **« SWimApiWaitSynchro »**.

L'application de vente obtient les messages du client monétique par l'appel de la fonction **« SWimApiGetMessage » (Réponse)**.

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

4. EXEMPLE D'ECHANGES

L'exemple, ci-après, décrit l'ensemble des appels aux fonctions de la librairie d'interface « **wcl_dll** » pour la réalisation d'une opération de débit par carte bancaire comprenant une validation par une hôtesse de caisse.

étape	action	Appel à
1	Démarrage du client monétique	« SWimApiOpen »
2	Émission de la requête de débit	« SWimApiSendMessage »
3	Écoute du client monétique	« SWimWaitSynchro »
4	Réception de la demande de validation	« SWimApiGetMessage »
5	Émission de la touche frappée	« SWimApiSendMessage »
3	Écoute du client monétique	« SWimWaitSynchro »
4	Réception de la réponse	« SWimApiGetMessage »

5. FORMATAGE DES REQUETES ET DES REPONSES

5.1. FORMAT

Les messages échangés par l'application de vente et le client monétique sont transportés en ASCII dans un format propriétaire :

➔ Un message est constitué d'une suite d'informations mises bout à bout.

➔ Toute information est encodée par une étiquette suivie de la valeur de l'information.

Les étiquettes sont représentées par des chaînes entre accolades : **{Etiquette}**

Les valeurs sont localisées entre l'accolade fermante de l'étiquette associée et l'accolade ouvrante de l'étiquette suivante : **}Valeur{**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

Deux **informations** sont systématiquement présentes dans tout **message** :

Code service du message	**Etiqueté {SV}**
Code commande du message	**Etiqueté {CD}**

5.2. EXEMPLE DE MESSAGE

Exemple

{SV}s0{CD}c2g{SS]R{PS}1{F} ***/ requête Login pour le poste 1 /***

*NB1 : Les étiquettes **{SS} et {F}** sont mises en place automatiquement par la DLL (**wcl_dll.cll**) lors de l'appel à la fonction **SWimApiSendMessage**.*

*NB2 : l'étiquette **{PS}** est ajoutée automatiquement si elle est absente – elle est valorisée avec la **valeur de IdStation** de la fonction **SwimApiOpen***

5.3. CONSTRUCTION D'UNE REQUETE

Les messages sont construits par l'application de vente par l'appel des fonctions ci-après de la librairie d'interface :

« ATAG_WriteInit »	**Initialisation de la formation du message**
« ATAG_SetStringVal »	**Ajout d'une information au message**

La construction du message débute par l'appel de la fonction **« ATAG_WriteInit »**.

La construction du message se poursuit par des appels successifs de la fonction **« ATAG_SetStringVal »**.

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

5.4. EXTRACTION DES INFORMATIONS D'UNE REPONSE

Les informations sont extraites d'un message par l'application de vente par l'appel des fonctions ci-après de la librairie d'interface :

« ATAG_ReadInit »	**Initialisation de l'extraction des informations du message**
« ATAG_GetStringVal »	**Extraction d'une information du message**

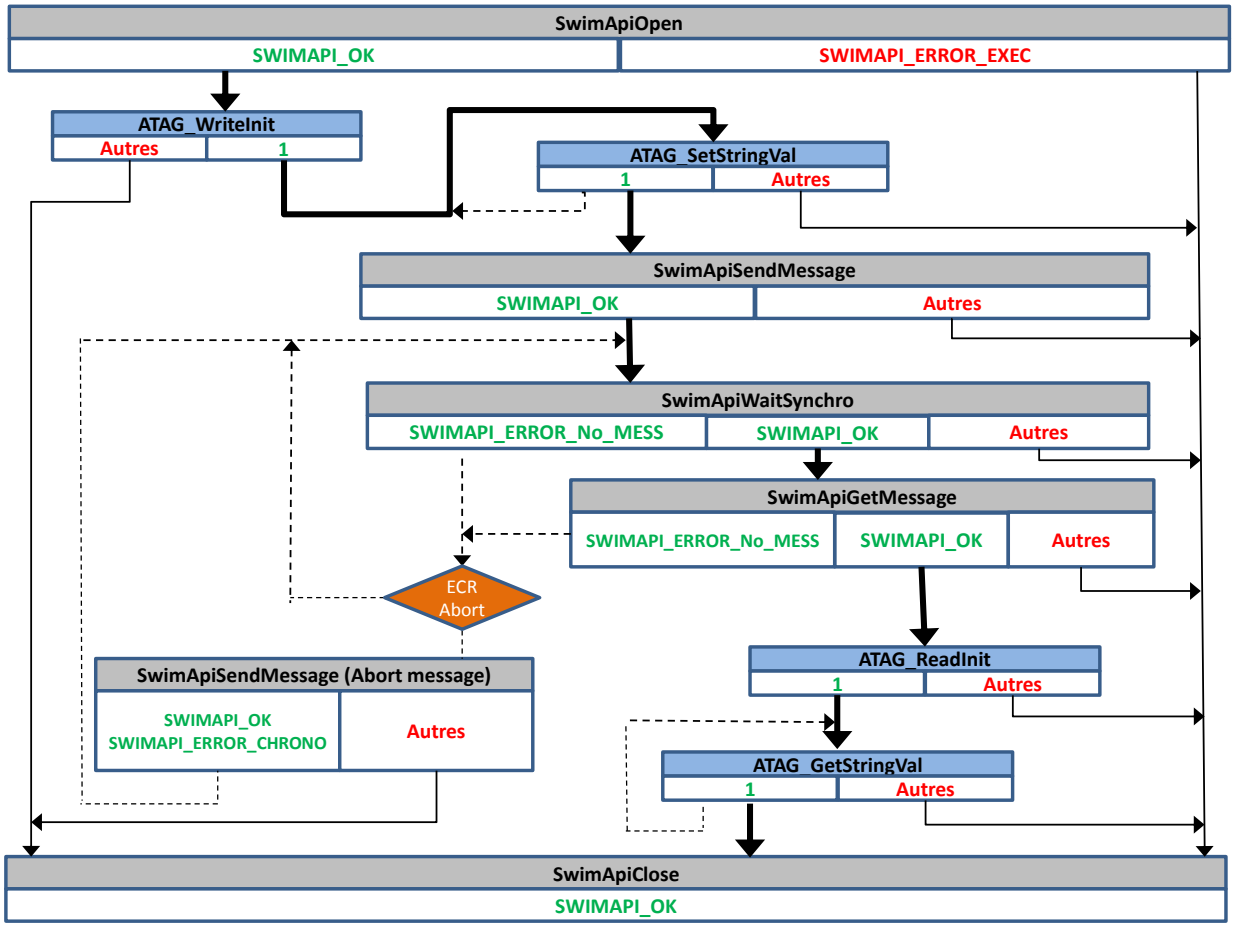
L'extraction des informations du message débute par l'appel de la fonction **« ATAG_ReadInit »**.

L'extraction des informations du message se poursuit par des appels successifs de la fonction **« ATAG_GetStringVal »**.

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

SCHEMA TYPE DE L'INTERFACE MONETIQUE

1. SCHEMA



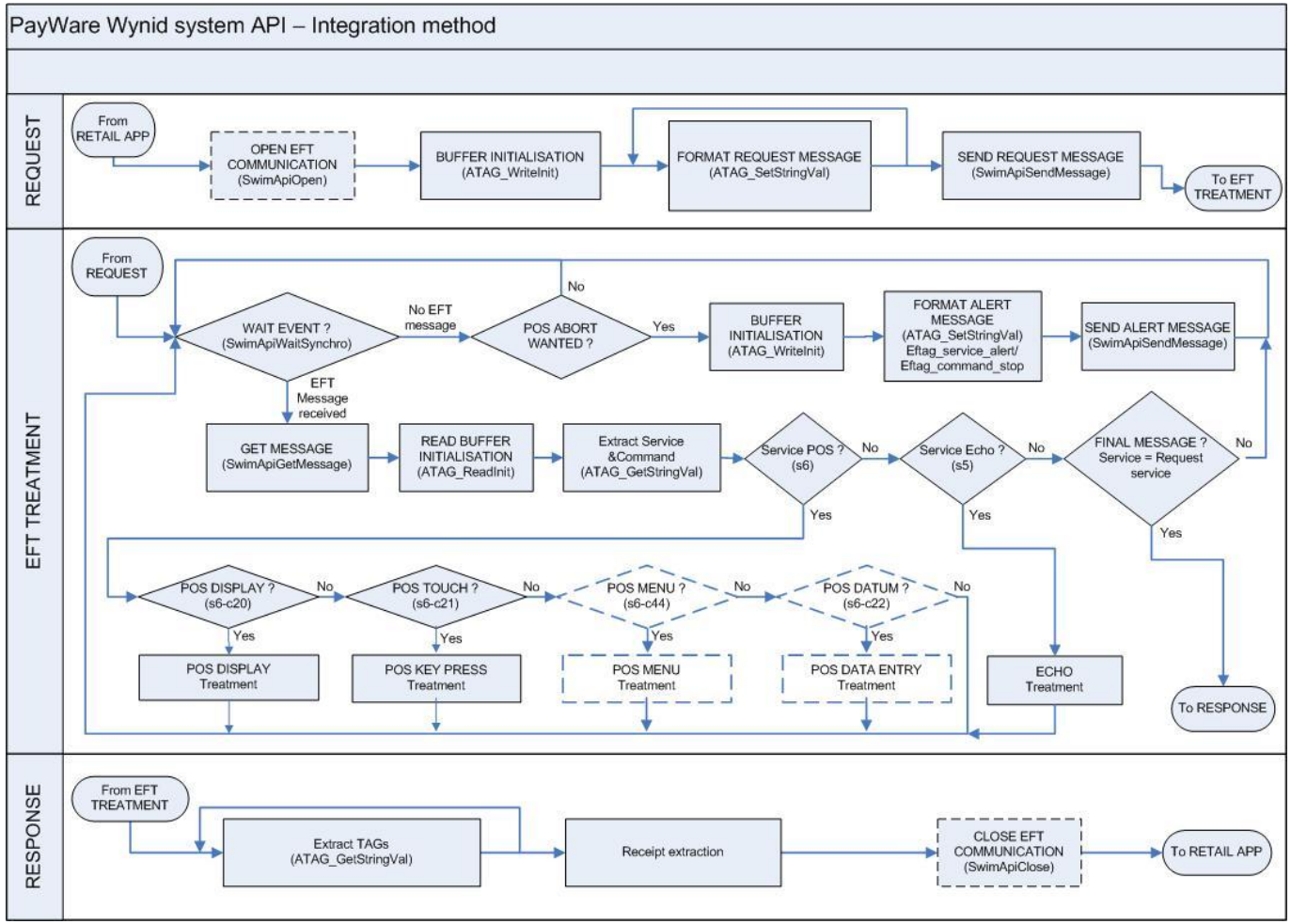
© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

Page blanche laissée intentionnellement

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

CINEMATIQUE DE L'INTERFACE MONETIQUE

1. CINEMATIQUE



© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

Page blanche laissée intentionnellement

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

IDENTIFICATION DES TRACES

La génération des traces est dépendantes des paramètres passés lors de l'appel de la fonction **SwimApiOpen** et des paramètres du fichier **.\WPAF\wclop_00.wt.**

Exemple

Extrait des paramètres **SwimApiOpen**

```
SWimApiOpen_
##### WCL_DLL Version 8.2.2.0 ####################################################
ux_LogsLevel      : 1                                   // Ldebug=niveau de trace de la DLL (0 ou 1)
cpx_LogsFileName  : .\WLOG\CLIENT.TXT                   // localisation des traces et nom du fichier trace client
ux_LogsSize       : 4096                                // taille maximum de la trace
ux_PosIdentifier  : 1                                   // IdStation=numéro du poste par défaut (si non défini par tag PS)
cpx_ClientPath    : C:\wynid \CLIENT\mcl_71.exe         //localisation et nom du client monétique
################################################################################
```

Extrait des paramètres **.\WPAF\wclop_00.wt**

```
/* ((((( Taille d'archivage des traces DLL ))))))))))))))))))))))))))))))))))))))) */
[LogSize:4096]
```


© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

1. TRACES DLL

Les traces de l'activité de la librairie dynamique **wcl_dll.dll** sont générées (si le paramètre **Ldebug** est fixé à 1 dans la fonction **SwimApiOpen**).

Ces traces sont localisées dans le répertoire défini pour les traces du module client défini lors de l'appel de la fonction **SwimApiOpen**

Dans l'exemple ci-dessus, les traces seront situées dans : **C:\wynid \CLIENT\WLOG**, où :

clientpath= C:\wynid \CLIENT + logfilepath=.\WLOG

Ces traces sont nommées :

« WCL_DLL_0.LOG »	Enregistre les paramètres d'appel (SwimApiOpen) et de fermeture (SwimApiclose)
« WCL_DLL_XXX.LOG »	Enregistre l'activité de la librairie **XXX=identifiant de la Station** (**IdStation** de la fonction **SwimApiOpen** ou valeur du tag **PS**)

Chacune de ces traces est limitée à la taille définie par le paramètre **LogSize** du fichier **.\WPAF\wclop_00.wt**

En cas de dépassement, une **archive est générée** et nommée **« WCL_DLL_0.LOB »** et **« WCL_DLL_XXX.LOB »**.

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

2. TRACES MCL

Le module client **MCL_71** génère des traces selon le paramètre spécifié dans la fonction **SwimApiOpen**.

Ces traces sont localisées dans le répertoire défini pour les traces du module client défini lors de l'appel de la fonction **SwimApiOpen**

Dans l'exemple ci-dessus, les traces seront situées dans : **C:\wynid \CLIENT\WLOG**, où : **(clientpath= C:\wynid \CLIENT + logfilepath=.\WLOG)**

Ces traces sont nommées avec la racine du nom de fichier défini lors de l'appel de la fonction **SwimApiOpen**

Dans l'exemple ci-dessus le nom du fichier trace est **CLIENT.TXT**, les traces seront nommées :

« CLIENT.POS »	Enregistre les échanges entre le système de caisse **(POS)** et le client monétique
« CLIENT.SRV »	Enregistre les échanges entre le client monétique et le serveur monétique **(SRV)**

Chacune de ces traces est limitée à la taille définie par le paramètre **LogSize** défini lors de l'appel de la fonction **SwimApiOpen**.

En cas de dépassement, une **archive est générée** et nommée **« CLIENT_POS.BAK»** et **« CLIENT_SRV.BAK »**.

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

Page blanche laissée intentionnellement

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

FONCTIONS D'ECHANGES AVEC LE CLIENT MONETIQUE

1. FONCTION « SWIMAPIOPEN »

1.1. FONCTIONNALITES

La fonction « `SWimApiOpen` » permet de démarrer l'exécutable client monétique.

1.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
IdInstance	Adresse de l'identifiant de l'instance client monétique démarrée
TypeClient	TOUJOURS VALORISÉ À 1
ModeEvent	TOUJOURS VALORISÉ À 1
ModeInterface	TOUJOURS VALORISÉ À 0
Ldebug	TOUJOURS VALORISÉ À 1
FileTrace	Répertoire et nom générique des fichiers générés par l'interface monétique pour tracer les échanges
ModuloTrace	Taille maximale d'un fichier de traces (en Ko)
IdStation	Numéro du point d'acceptation
Lang	TOUJOURS VALORISÉ À 250 = Français
Path	Répertoire hébergeant l'exécutable client monétique
ModeEcho	Type de retransmission des affichages et saisies effectués sur le lecteur monétique
TimeOutMaxMessage	VALEUR PRÉCONISÉE À 20 (s)
ModeOffline	TOUJOURS VALORISÉ À 0
PathServeur	TOUJOURS VALORISÉ À NULL
Scellement	TOUJOURS VALORISÉ À 0
PathMemoireDouble	TOUJOURS VALORISÉ À NULL
NbJoursSauves	TOUJOURS VALORISÉ À 0

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

1.2.1. PARAMETRE « *IdInstance* »

Paramètre « *IdInstance* »	
Type	Pointeur sur un entier non signé
Objet	Stocker la valeur d'identification du process client démarré. La valeur stockée sera ensuite passée en paramètres des fonctions : ▪ « **SWimApiClose** », ▪ « **SWimApiSendMessage** », ▪ « **SWimApiWaitSynchro** » et ▪ « **SWimApiGetMessage** ».
Pré-requis de valorisation	
Exemple de valorisation	

1.2.2. PARAMETRE « *FileTrace* »

Paramètre « *FileTrace* »	
Type	Pointeur sur chaîne de caractères
Objet	Spécifier le répertoire destinataire et le nom générique des fichiers générés par l'interface monétique pour tracer les échanges. En fait, deux fichiers sont générés : « **xxx.POS** » et « **xxx.SRV** » où **XXX** représente le nom générique. Ces fichiers sont renommées « **xxx_POS.BAK** » et « **xxx_SRV.BAK** » dès lors que leur taille excède la limite valorisée par le paramètre « **ModuloTrace** ».
Pré-requis de valorisation	
Exemple de valorisation	Soit le répertoire destinataire suivant : **« C:\WYNID\WCL\WLOG »** Soit le nom générique suivante : **« CLIENT »** ▪ Le paramètre « *FileTrace* » devra alors être valorisé comme suit : **« C:\WYNID\WCL\WLOG\CLIENT »** ▪ Les fichiers de traces générés seront alors : ○ « **CLIENT.POS** », ○ « **CLIENT.SRV** », ○ « **CLIENT_POS.BAK** » et ○ « **CLIENT_SRV.BAK** ».

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

1.2.3. PARAMETRE « *ModuloTrace* »

Paramètre « *ModuloTrace* »	
Type	Entier non signé (exprimé en Ko)
Objet	Spécifier la taille limite d'un fichier de traces
Pré-requis de valorisation	**Il est impératif de prévoir une taille limite supérieure à 1Mo.** **Valeur conseillée : 4Mo**
Exemple de valorisation	

1.2.4. PARAMETRE « *IdStation* »

Paramètre « *IdStation* »	
Type	Entier non signé
Objet	Spécifier le numéro de caisse
Pré-requis de valorisation	**Il est impératif que ce paramètre soit valorisé de manière unique pour chacun des postes se connectant sur un même serveur monétique pour un même Ilot**
Exemple de valorisation	

1.2.5. PARAMETRE « *Path* »

Paramètre « *Path* »	
Type	Pointeur sur chaîne de caractères
Objet	Spécifier le répertoire hébergeant l'exécutable « **MCL_71.EXE** » et ses répertoires associés.
Pré-requis de valorisation	
Exemple de valorisation	Soit le répertoire hébergeant suivant : **« C:\WYNID\WCL »**. ▪ Le paramètre « *Path* » devra alors être valorisé comme suit : **« C:\WYNID\WCL »**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

1.2.6. PARAMETRE « *ModeEcho* »

Paramètre « *ModeEcho* »	
Type	Entier non signé
Objet	Spécifier le type de retransmission des affichages et saisies effectués sur le lecteur monétique. Cette fonctionnalité n'est à activer que dans les cas suivants : ▪ le lecteur monétique ne possède pas d'afficheur dédié, ▪ l'application de vente souhaite reproduire sur son afficheur les interfaces avec le porteur de carte.
Pré-requis de valorisation	**Les valeurs autorisées sont les suivantes :** 0 → Retransmission désactivée (valeur conseillée) 1 → Retransmission des affichages du lecteur monétique
Exemple de valorisation	

1.3. CODES RESULTAT

CODE RESULTAT	VALEUR	DESCRIPTION
SWIMAPI_OK	**0**	**Démarrage du client monétique effectué**
SWIMAPI_ERROR_EXEC	**1**	**Démarrage du client monétique impossible**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

2. FONCTION « SWIMAPICLOSE »

2.1. FONCTIONNALITES

La fonction « **SWimApiClose** » permet d'arrêter l'exécutable client monétique.

Important	**Il est impératif d'appeler cette fonction après l'obtention du code retour « SWIMAPI_ERROR_EXEC » lors de l'appel de la fonction « `SWimApiOpen` ».**

2.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
`IdInstance`	**Identifiant de l'instance client monétique démarrée**
`TypeClient`	**TOUJOURS VALORISÉ À 0**

2.3. CODES RESULTAT

CODE RESULTAT	VALEUR	DESCRIPTION
SWIMAPI_OK	**0**	**Arrêt du client monétique effectué**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

3. FONCTION « SWIMAPISENDMESSAGE »

3.1. FONCTIONNALITES

La fonction « **SWimApiSendMessage** » permet d'émettre un message vers le client monétique.

3.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
IdInstance	Identifiant de l'instance client monétique démarrée *(Se référer au paragraphe 0)*
IdService	Code service du message à émettre
LPZBuffer	Adresse de stockage du message à émettre
LgBuf	Adresse de stockage de la taille maximale de stockage du message à émettre

3.2.1. PARAMETRE « *IDSERVICE* »

Paramètre « *IdStation* »	
Type	Pointeur sur chaîne de caractères
Objet	Indiquer à l'interface monétique le code service contenu dans le message à émettre.
Pré-requis de valorisation	**Il est impératif que la chaîne passée soit identique à celle contenue dans le message sous l'étiquette « EFTAG_HEAD_SERVICE » ({SV})**
Exemple de valorisation	

3.2.2. PARAMETRE « *LPZBUFFER* »

Paramètre « *LPZBuffer* »	
Type	Pointeur sur chaîne de caractères
Objet	Fournir le message à émettre.
Pré-requis de valorisation	**Le contenu du message doit être construit par l'appel des fonctions « ATAG_WriteInit » et « ATAG_SetStringVal ».**
Exemple de valorisation	

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

3.2.1. PARAMETRE « *LgBuf* »

Paramètre « *LgBuf* »	
Type	Pointeur sur un entier non signé
Objet	Spécifier la taille du message à émettre.
Pré-requis de valorisation	
Exemple de valorisation	

3.3. CODES RESULTAT

CODE RESULTAT	VALEUR	DESCRIPTION	RETAIL ACTION
SWIMAPI_OK	**0**	Émission effectuée, acquittement de prise en compte reçu	OK
SWIMAPI_ERROR_CHRONO	**6**	Émission effectuée, acquittement d'impossibilité d'abandon à ce stade reçu **Note** : Ce code résultat ne peut s'obtenir que lors de l'émission d'une requête d'abandon monétique	Message ignoré. **Ne pas quitter le process**.
SWIMAPI_ERROR_SERV	**5**	Émission effectuée, acquittement d'indication d'une opération déjà en cours reçu	Message ignoré. **Ne pas quitter le process**
SWIMAPI_ERROR_EXEC	**1**	Émission échouée ➔ Client monétique arrêté	Incident technique. **Quitter le process et réparer**
SWIMAPI_ERROR_SYNT1 **SWIMAPI_ERROR_SYNT2**	**3** **4**	Émission échouée ➔ Syntaxe du message incorrecte	Incident technique. **Quitter le process et réparer**
SWIMAPI_ERROR_INTERNAL	**8**	Émission échouée ➔ Erreur fatale	Incident technique. **Quitter le process et réparer**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

3.3.1. ACTIONS A ENTREPRENDRE

L'obtention des codes résultat **« SWIMAPI_ERROR_CHRONO » et « SWIMAPI_ERROR_SERV »** ne doit pas entraîner l'arrêt du dialogue en cours.

En effet, ces codes s'obtiennent lorsqu'une opération est en cours et qu'une tentative d'émission d'une nouvelle requête intervient avant la réception de la réponse à l'opération en cours.

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

Page blanche laissée intentionnellement

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

4. FONCTION « SWIMAPIWAITSYNCHRO »

4.1. FONCTIONNALITES

La fonction « **SWimApiWaitSynchro** » permet d'attendre la disponibilité d'un message par client monétique.

4.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
IdInstance	**Identifiant de l'instance client monétique démarrée** ***(Se référer au paragraphe 0)***
TimeOut	**Délai d'attente de disponibilité d'un message émis par le client monétique**

4.2.1. PARAMETRE « *TIMEOUT* »

Paramètre « *TimeOut* »	
Type	Entier signé
Objet	Exprimer, en millisecondes, le délai maximal d'attente de disponibilité d'un message émis par le client monétique. (valeur conseillée 200 ms)
Pré-requis de valorisation	
Exemple de valorisation	

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

4.3. CODES RESULTAT

CODE RESULTAT	VALEUR	DESCRIPTION
SWIMAPI_OK	**0**	Message émis par le client monétique disponible
SWIMAPI_ERROR_NO_MESS	**7**	Aucun message émis par le client monétique disponible
SWIMAPI_ERROR_EXEC	**1**	Attente échouée ➔ Client monétique arrêté
SWIMAPI_ERROR_INTERNAL	**8**	Attente échouée ➔ Erreur fatale

4.3.1. ACTIONS A ENTREPRENDRE

L'obtention du code résultat **« SWIMAPI_ERROR_NO_MESSAGE »** ne doit pas entraîner l'arrêt du dialogue en cours.

Une nouvelle attente de disponibilité d'un message émis par le client monétique devra être lancée, éventuellement après des traitements locaux à l'application de vente.

Exemple	*Détection de l'appui sur une touche du clavier de l'opérateur*

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

5. FONCTION « SWIMAPIGETMESSAGE »

5.1. FONCTIONNALITES

La fonction « **SWimApiGetMessage** » permet de recevoir un message émis par client monétique.

5.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
IdInstance	Identifiant de l'instance client monétique démarrée *(Se référer au paragraphe 0)*
IdService	Code service du message reçu (émis par le client monétique)
LPZBuffer	Adresse de stockage du message reçu (émis par le client monétique)
LgBuf	Adresse de stockage de la taille maximale de stockage du message reçu (émis par le client monétique)

5.2.1. PARAMETRE « *IDSERVICE* »

Paramètre « *IdStation* »	
Type	Pointeur sur chaîne de caractères
Objet	Obtenir de l'interface monétique le code service contenu dans le message reçu.
Pré-requis de valorisation	
Exemple de valorisation	

5.2.2. PARAMETRE « *LPZBUFFER* »

Paramètre « *LPZBuffer* »	
Type	Pointeur sur chaîne de caractères
Objet	Mémoriser le message reçu
Pré-requis de valorisation	**Les messages reçu pouvant être de taille conséquente, il est impératif que la taille de la zone mémoire réservée soit supérieure ou égale à 20Ko + 1 octet.**
Exemple de valorisation	

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

5.2.3. PARAMETRE « *LgBuf* »

Paramètre « *LgBuf* »	
Type	Pointeur sur un entier non signé
Objet	Connaître la taille exacte du message émis par le client monétique
Pré-requis de valorisation	
Exemple de valorisation	

5.3. CODES RESULTAT

CODE RESULTAT	VALEUR	DESCRIPTION
SWIMAPI_OK	**0**	Message émis par le client monétique disponible
SWIMAPI_ERROR_NO_MESS	**7**	Aucun message émis par le client monétique disponible
SWIMAPI_ERROR_EXEC	**1**	Attente échouée ➔ Client monétique arrêté
SWIMAPI_ERROR_INTERNAL	**8**	Attente échouée ➔ Erreur fatale

5.3.1. ACTIONS A ENTREPRENDRE

L'obtention du code résultat **« SWIMAPI_ERROR_NO_MESSAGE »** ne doit pas entraîner l'arrêt du dialogue en cours uniquement dans le cas où l'appel à la fonction « **SWimApiWaitSynchro** » a fourni le code résultat **« SWIMAPI_OK »**.

Autrement, une nouvelle attente de disponibilité d'un message émis par le client monétique devra être lancée, éventuellement après des traitements locaux à l'application de vente.

Exemple	*Détection de l'appui sur une touche du clavier de l'opérateur*

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

Page blanche laissée intentionnellement

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

FONCTIONS DE CREATION D'UN MESSAGE

1. FONCTION « ATAG_WRITEINIT »

1.1. FONCTIONNALITES

La fonction « **ATAG_WriteInit** » permet de mémoriser les caractéristiques de la zone de stockage du message à émettre vers le client monétique.

1.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
BufferToBuild	Adresse de stockage du message à émettre vers le client monétique
MaxLengthToBuild	Taille maximale de stockage du message à émettre vers le client monétique

1.2.1. PARAMETRE « *BUFFERTOBUILD* »

Paramètre « *BufferToBuild* »	
Type	Pointeur sur une chaîne de caratères
Objet	Mémoriser l'adresse mémoire à laquelle sera construit le message à émettre vers le client monétique.
Pré-requis de valorisation	**Les messages à émettre vers le client monétique pouvant être de taille conséquente, il est impératif que la taille de la zone mémoire allouée soit supérieure ou égale à 10Ko + 1 octet.**

1.2.2. PARAMETRE « *MAXLENGTHTOBUILD* »

Paramètre « *MaxLengthToBuild* »	
Type	Entier non signé
Objet	Spécifier la taille de la zone mémoire allouée pour le stockage du message à émettre vers le client monétique

1.3. CODES RESULTAT

VALEUR	DESCRIPTION
1	**Mémorisation effectuée**
Autres	**Mémorisation échouée**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

2. FONCTION « ATAG_SETSTRINGVAL »

2.1. FONCTIONNALITES

La fonction « **ATAG_SetStringVal** » permet d'ajouter une donnée ***{TAG}valeur*** dans la zone de stockage du message à émettre vers le client monétique.

2.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
`TagToAdd`	Adresse de la chaîne de caractères identifiant l'étiquette ***{TAG}*** de la donnée à ajouter dans le message à émettre
`ASCIIValueToAdd`	Adresse de la chaîne de caractères valorisant la donnée ***}valeur*** à ajouter dans le message à émettre

2.3. CODES RESULTAT

VALEUR	DESCRIPTION
1	**Ajout effectuée**
Autres	**Ajout échouée**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

FONCTIONS D'ANALYSE D'UN MESSAGE

1. FONCTION « ATAG_READINIT »

1.1. FONCTIONNALITES

La fonction « **ATAG_ReadInit** » permet de mémoriser les caractéristiques de la zone de stockage du message émis par le client monétique.

1.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
`BufferToExtract`	**Adresse de stockage du message émis par le client monétique**
`MaxLengthToExtract`	**Taille maximale de stockage du message émis par le client monétique**

1.2.1. PARAMETRE « *BUFFERTOEXTRACT* »

Paramètre « *BufferToExtract* »	
Type	Pointeur sur une chaîne de caractères
Objet	Mémoriser l'adresse mémoire à laquelle sera construit le message émis par le client monétique.

1.2.2. PARAMETRE « *MAXLENGTHTOEXTRACT* »

Paramètre « *MaxLengthToExtract* »	
Type	Entier non signé
Objet	Spécifier la taille du message émis par le client monétique
Pré-requis de valorisation	**La valeur à affecter à ce paramètre est celle obtenue dans le paramètre « `LgBuf` » lors de l'appel de la fonction « `SWimApiGetMessage` ».**

1.3. CODES RESULTAT

VALEUR	DESCRIPTION
1	**Mémorisation effectuée**
Autres	**Mémorisation échouée**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

2. FONCTION « ATAG_GETSTRINGVAL »

2.1. FONCTIONNALITES

La fonction « **ATAG_GetStringVal** » permet d'extraire une donnée de la zone de stockage du message émis par le client monétique.

2.2. PARAMETRES D'APPEL DE LA FONCTION

PARAMETRE	OBJET
`TagToExtract`	**Adresse de la chaîne de caractères identifiant l'étiquette de la donnée à extraire**
`ExtractedASCIIValue`	**Adresse de stockage de la chaîne de caractères valorisant la donnée à extraire**
`SearchFromStart`	**Indicateur de recherche de l'information à partir de l'origine du message**

2.2.1. PARAMETRE « *TAGTOEXTRACT* »

Paramètre « *TagToExtract* »	
Type	Pointeur sur une zone mémoire
Objet	Spécifier l'adresse de la chaîne de caractères contenant l'étiquette de la donnée à extraire

2.2.2. PARAMETRE « *EXTRACTEDASCIIVALUE* »

Paramètre « *ExtractedASCIIValue* »	
Type	Pointeur sur une zone mémoire
Objet	Spécifier l'adresse de stockage de la chaîne de caractères valorisant la donnée à extraire
Pré-requis de valorisation	**Il est impératif que la zone mémoire allouée soit correctement dimensionnée par rapport à la taille attendue de la valeur à extraire.** **Pour éviter tout risque de débordement mémoire, il est souhaitable de dimensionner cette zone mémoire de la même façon que la zone de stockage du message émis par le client monétique.**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

2.2.3. PARAMETRE « *SEARCHFROMSTART* »

Paramètre « *SearchFromStart* »	
Type	Entier
Objet	Spécifier si la donnée à extraire est à rechercher depuis l'origine du message émis par le client monétique ou depuis la position courante d'extraction
Pré-requis de valorisation	**Ce paramètre est à valoriser à 1 dans tous les cas de recherche d'une donnée non reproductible.** **Dans le cas de recherche d'une donnée reproductible, la première occurrence est à rechercher en valorisant ce paramètre à 1, les autres étant à rechercher en valorisant ce paramètre à 0.**

2.3. CODES RESULTAT

VALEUR	DESCRIPTION
1	**Extraction effectuée**
Autres	**Extraction échouée**

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.

Page blanche laissée intentionnellement

© 2016 VeriFone. All rights reserved. No portion of this document may be reproduced or distributed in any form or by any means without the prior written permission of said company. All trademarks are the property of their respective owners.